PROJECT AND TEAM INFORMATION

Project Title

Inverted Index and Code-Based Search Engine

Student / Team Information

Team Name: *Team #*	*Code Acer* *DSCPP-III-2026-T112*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Gupta, Anmol – 2510385219* *anmolgupta1523@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Kumar, Harshit – 2510012451* *harshitkumarlearning@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Goyal, Atharva – 2510360047* *atharvagoyal45@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Nainwal, Navneet – 2510010627* *nainwalnavneet@gmail.com* 

PROPOSAL DESCRIPTION

Motivation (1 pt)

In today's digital world, the amount of information stored in files, documents, source-code repositories, and databases is increasing rapidly. Finding relevant information from a large collection of files using traditional methods can be slow because every search may require scanning files one by one. This is especially challenging for large codebases, where developers need to quickly locate functions, variables, classes, keywords, or specific code patterns. Therefore, an efficient search mechanism is required to retrieve relevant information quickly.

The Inverted Index and Code-Based Search Engine aims to develop a fast and efficient search system for text and source-code files. An inverted index maps keywords to the files and locations where they occur. Instead of scanning every file for each query, the system directly accesses the index, significantly reducing search time.

The project focuses on applying Data Structures and Algorithms concepts to build a practical search engine. It can use structures such as hash tables for indexing and fast keyword lookup, linked lists for maintaining document references, trees for organized searching, and sorting and searching algorithms for ranking and retrieving results. For source-code files, the system can additionally identify programming-specific elements such as functions, variables, classes, and keywords.

The main problem addressed by this project is therefore how to efficiently search and retrieve relevant information from a large collection of text and code files while minimizing search time and unnecessary file scanning. The proposed system aims to provide faster keyword-based searching, organized results, and a foundation for scalable code-search applications. It also demonstrates how fundamental DSA concepts can be combined to solve a real-world information retrieval problem.

State of the Art/Current solution

Currently, modern search systems such as Google, Elasticsearch, and Apache Lucene use indexing techniques to provide fast and efficient information retrieval. A common approach is the inverted index, which stores keywords along with the documents and locations where they occur. This allows the system to retrieve relevant results without scanning every file for each search query.

Before indexing, documents are processed through tokenization, normalization, filtering, and sometimes stemming to improve search accuracy. Search engines also use ranking techniques such as TF-IDF and BM25 to determine the relevance of results. During a search, the query is matched against the index instead of scanning every document, which makes retrieval much faster.

For source-code searching, platforms such as GitHub and modern IDEs provide features for searching keywords, functions, classes, and variables across code files. These systems are powerful and optimized for large-scale projects, but their internal implementations can be complex.

Our project proposes a simplified code-based search engine built from scratch. It focuses on understanding the fundamental concepts behind modern search systems by implementing inverted indexing, hashing, searching, sorting, and document retrieval using DSA techniques. This provides a practical and educational approach to efficient code searching.

Project Goals and Milestones

Goal:

To develop a fast and simple code-based search engine that uses an inverted index to efficiently search keywords across multiple source-code files.

Initial Milestones:

- *Create an inverted index for keywords and files.*
- *Implement fast searching using hashing and searching algorithms.*
- *Search source-code files such as C++, Java,8and Python.*
- *Display matching files and code locations.*
- *Apply DSA concepts such as hashing, linked lists, trees, sorting, and searching.*

Milestones:

1. *Analyze requirements and design the system.*
2. *Read and process source-code files.*
3. *Build the inverted index.*
4. *Implement the search and sorting modules.*
5. *Develop a basic user interface.*
6. *Test and optimize the system.*

Project Approach (3 pts)

The project will be developed by first collecting and processing multiple source-code files. The system will extract important keywords and create an inverted index that maps each keyword to the files and locations where it occurs.

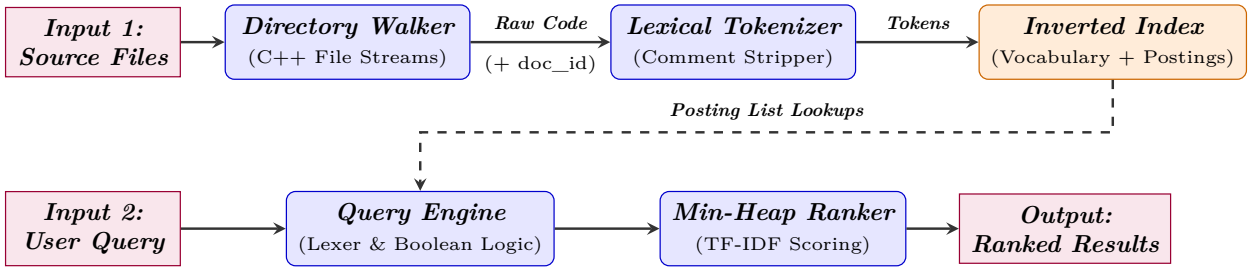
The search module will use the index to quickly find matching keywords without scanning every file. Hashing, linked lists, trees, searching, and sorting algorithms will be used to improve efficiency and organize the results.

A simple user interface will allow users to enter a keyword and view the relevant files and code locations. The system will be tested using different source-code files and optimized for faster searching.

Technologies & Platforms:

C++ *– Core logic and DSA implementation*
HTML, CSS, JavaScript *– Basic user interface*
VS Code *– Development environment*
Windows/Linux *– Execution and testing platform*

System Architecture (High Level Diagram)



Data Flow Interaction:

Data Ingestion Pipeline: The workflow starts with **Input 1 (Source Files)**, which are read by the **Directory Walker** using C++ file streams. This walker sends the raw code and assigned `doc_id` values to the **Lexical Tokenizer**. The tokenizer strips out comments and noise, generating clean tokens that populate the **Inverted Index** (storing both the vocabulary and corresponding postings).

Query Processing: When a search is initiated via **Input 2 (User Query)**, it routes directly to the **Query Engine**. This engine applies a lexer and boolean logic to perform **Posting List Lookups** against the previously built Inverted Index.

Ranking and Output: The retrieval results are then passed to the **Min-Heap Ranker**, which applies TF-IDF scoring to evaluate and sort document relevance. The final sorted list is presented to the user as **Output (Ranked Results).**

Project Outcome/Deliverables

The project will deliver a standalone, high-performance CLI-based search engine written in C/C++ for indexing and searching local source-code repositories.

Key deliverables:

- *A Word Inverted Index with dynamic posting lists for fast keyword lookup.*
- *Efficient memory management using malloc() and free().*
- *Fast multi-keyword retrieval without repeated linear file scanning.*
- *Support for Boolean queries (AND, OR, NOT) using posting-list operations.*
- *Top-K relevance-based ranking of matching files using a scoring mechanism and priority queue.*
- *Binary index persistence to save and restore the pre-computed index without re-indexing.*
- *Search support for keywords, functions, variables, and classes.*
- *Complete source code, documentation, and test cases for evaluating search efficiency and accuracy.*

The project demonstrates the practical application of DSA concepts such as hashing, linked lists, pointers, recursion, sorting, searching, and priority queues to build an efficient real-world code search engine.

Assumptions

- *Source File Format: The engine assumes all indexed targets are text-readable source code files (such as .c, .cpp, .h, .java, .py, .txt), ignoring binary, compiled, or executable formats (.o, .exe, .bin).*
- *Character Encoding and Identifiers: Code identifiers, keywords, and search queries are assumed to use standard ASCII alphanumeric characters and underscores(_), with non-alphanumeric symbols treated as token delimiters.*
- *System Memory Capacity: The combined vocabulary of unique keywords and associated posting lists is assumed to fit entirely within system RAM during runtime.*
- *File Stability During Ingestion: Target source files are assumed to remain static and unmodified on disk while the engine is actively reading, tokenizing, and constructing the in-memory inverted index.*
- *Case-Insensitive Ingestion: All tokens and search queries are assumed to be normalized to lowercase for uniform indexing and exact retrieval.*

References

- *Introduction to Information Retrieval – Manning, Raghavan, Schütze (Cambridge UP) (https://nlp.stanford.edu/IR-book/)*
- *Mastering Algorithms with C Kyle Loudon (O'Reilly Media).*
- *The C++ Programming Language (4th Edition) – Bjarne Stroustrup.*
- *Lucene/Elasticsearch Inverted Index Architecture & TF-IDF Retrieval Models. (https://www.elastic.co/guide/en/elasticsearch/reference/current/documents-indices.html)*